Algorithmic Trading and Quantitative Strategies

Part 11

Dr. Ayhan Yuksel, CFA, FRM, PRM

Robustness of a Trading Strategy

Dimensions of Robustness

- For a robust trading strategy, the resulting performance does not degrade significantly:
  - If parameters are **slightly** changed
  - If we run the strategy under a different regime (e.g. high volatility vs low volatility)
  - If we skip top *n* best trades
  - If we change the order types for entry/exit rules
  - If we slightly increase fees/slippage
  - Between in-sample vs out-of-sample periods

Statistical Testing for Trading Strategy Performance

Population vs Sample

- Assume that Y represents a collection of random variables $Y_1, Y_2, \ldots$ which are independent and identically distributed (i.i.d.), i.e. each random variable $Y_i \sim F(\Theta)$ has the same probability distribution (with parameter Θ) as the others and all are mutually independent.
- Our aim is to make inferences about true (and unobserved) population parameter Θ. And asssume that, from theory, there exists a certain sample statistics, θ, that is an unbiased estimater of true Θ, i.e. $\mathbb{E}[\theta] = \Theta$.
- Example
  - Assume that $Y_i \sim N(\mu, \sigma)$, where μ is the population mean, and σ is the population standard deviation
  - For a realization $Y_1, \ldots, Y_N$, sample mean defined as $m = \frac{1}{N} \sum Y_i$ is an unbiased estimater of μ:

$$\mathbb{E}[m] = \mathbb{E}[\frac{1}{N} \sum Y_i] = \frac{1}{N} \sum \mathbb{E}[Y_i] = \frac{1}{N} \sum \mu = \mu$$

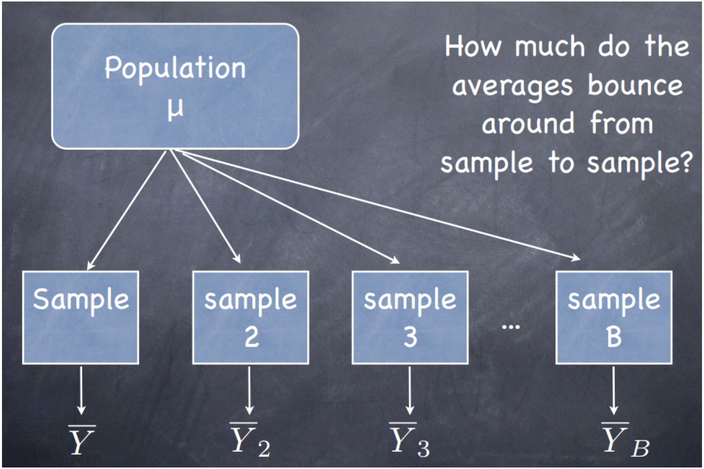
Sampling Distribution

- The sample mean m is itself a random variable, i.e. each time we draw a sample from the population, we get a new realization of m, possibly with a different value
- The distribution of this sample statistics, i.e. sample mean, is called **sampling distribution** of it
- Example:
  - If the population is Gaussian, i.e. $N(\mu, \sigma)$, construct the sampling distribution of sample mean m
  - We already know $\mathbb{E}[m] = \mu$
  - For the variance:

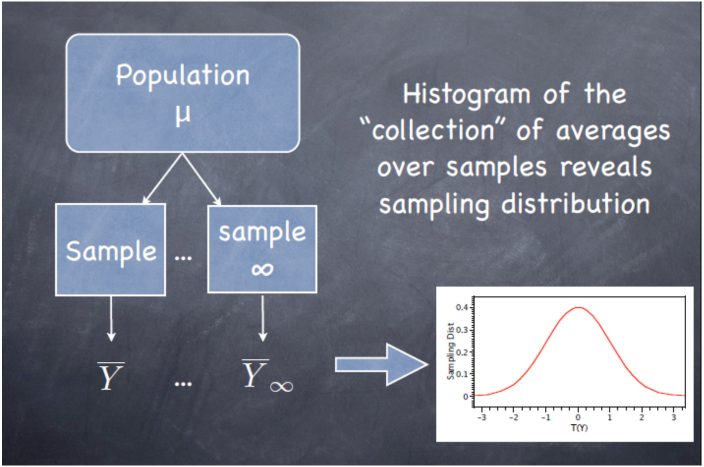
$$
\begin{aligned}
Var[m] &= \mathbb{E}[(m-\mu)^2] = \mathbb{E}[(\frac{1}{N}\sum Y_i - \mu)^2] \\
&= \frac{1}{N}\sum \mathbb{E}[(Y_i - \mu)^2] = \frac{1}{N}\sum \sigma^2 = \frac{\sigma^2}{N}
\end{aligned}
$$

  - Since m is just a sum of normally distributed variables, it has also normal distribution
  - Therefore $m \sim N(\mu, \frac{\sigma}{\sqrt{N}})$

Sampling Distribution



Sampling Distribution



Inference for Population Parameters

- Aim: Make inferences for the true population parameter after observing sample(s) from the population.
- If we can observe different samples from the population, then:
  - Take large number (say M) of samples from the population: $\{Y_1^1, \dots, Y_N^1\}; \{Y_1^2, \dots, Y_N^2\}; \dots; \{Y_1^M, \dots, Y_N^M\}$
  - Calculate test (i.e. sample) statistics for each sample: $TS(Y_1^1, \dots, Y_N^1), \dots, TS(Y_1^M, \dots, Y_N^M)$
    - e.g. sample mean, sample variance
  - Construct the **sampling distribution** of test statistics: i.e. G_{TS} is the distribution of TS's.
  - Construct confidence interval for test statistics: $Pr[G_{TS}^{-1}(\alpha/2) \leq TS \leq G_{TS}^{-1}(1-\alpha/2)] = 1-\alpha$
  - Invert test to calculate the confidence interval for the population parameter

Inference for Population Parameters

```r
# Assume a normally distributed population: N(mu=0.1, sigma=0.25)
M<-10000; N<-100
All.Sample.Stats<-vector()
for (i in 1:M){ #Sample M times
  mysample<-rnorm(N,0.1,0.25) #Sample length=N
  Sample.Stat<-mean(mysample) #Our sample stat is sample mean
  All.Sample.Stats<-c(All.Sample.Stats,Sample.Stat)
}

# Inference for population mean
mean(All.Sample.Stats) # Point estimate
```

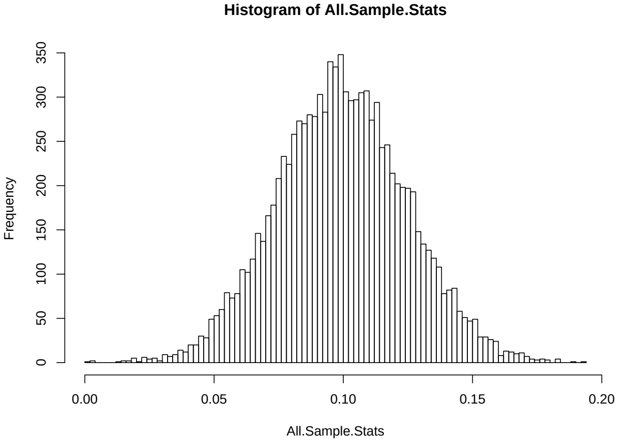
```
## [1] 0.09945006
```

```r
quantile(All.Sample.Stats,probs=c(0.05,0.95)) # Confidence interval
```

```
##         5%       95%
## 0.0581687 0.1414516
```

Inference for Population Parameters

```
# Construct sampling distribution
hist(All.Sample.Stats,100)
```



Inference for Population Parameters

- What if we can only observe one sample from the population
  - e.g. Our backtest results constitute just one realization (i.e. sample) from the true population of strategy returns. If we apply our strategy in another time period or for another asset, the returns will not be same and will be another sample from the true population
- If we observe only one sample, we can use two different methods to make inferences for the population parameters:
  - Parametric Approach
  - Bootstrap Approach

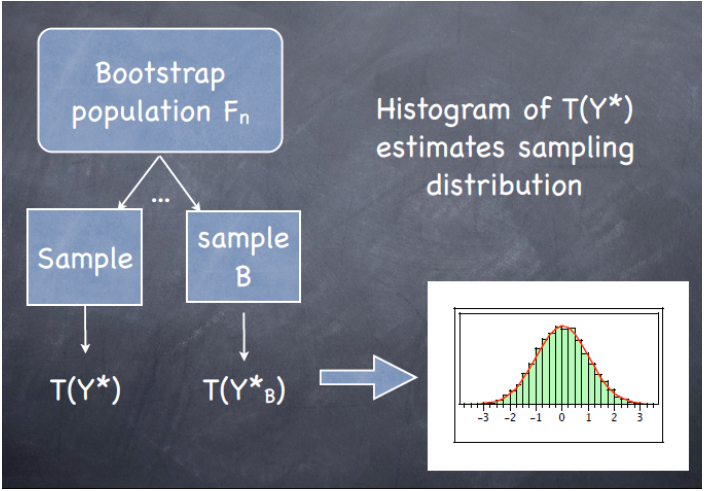
Parametric Approach

- In parametric approach, we make certain assumptions for the true population distributon.
- For example, if we assume that the true population distribution is Gaussian, $N(\mu, \sigma)$, then:
  - The sample mean is an unbiased estimator of population mean and the sample variance is an unbiased estimator of population variance
  - Thus we can make point estimates for μ and σ^2 using their sample counterparts
  - Continuing with the previous example, we know that the sampling distribution of sample mean is $m \sim N(\mu, \frac{\sigma}{\sqrt{N}})$
  - Thus we can construct confidence intervals such as $Pr[\mu - z_{\alpha/2}\frac{\sigma}{\sqrt{N}} \leq m \leq \mu + z_{\alpha/2}\frac{\sigma}{\sqrt{N}}] = 1 - \alpha$, where $z_{\alpha/2}$ is the quantile of standard normal distribution at confidence level $\alpha/2$
  - Construct the confidence interval for population parameter:$\mu \in m \pm z_{\alpha/2}\frac{\sigma}{\sqrt{N}}$
    - The true mean or proportion for the population exists and is a fixed number, but we don't know it
    - If we take a single sample, our single confidence interval may or may not include the population parameter.
    - However if we take many samples of the same size and create a confidence interval from each sample statistic, over the long run $1 - \alpha$ of our confidence intervals will contain the true population parameter.

Bootstrap Approach

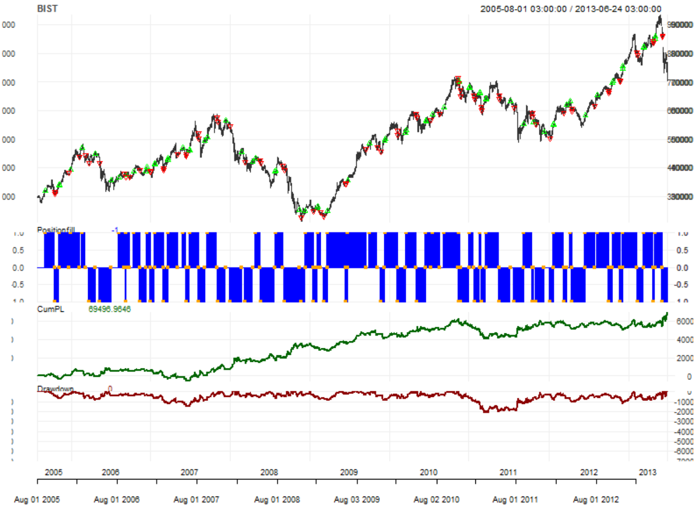
- Steps:
  - Bootstrap the returns: Randomly sample from strategy returns with replacement to generate a new simulated return series with same length as the original return series
  - Calculate the performance measures (e.g. sharpe ratio, max drawdown) for each bootstrap sample
  - Construct a sampling distribution for the performance metric
  - Calculate the confidence interval for the population value of performance metric

Bootstrap Approach



Bootstrap Approach

Example: Our Previous MA Crossover Strategy



Bootstrap Approach

```r
# Fnc for max drawdown
Max.Drawdown<-function(myzoo){
  min(myzoo/runMax(myzoo, n=1,cumulative =T)-1,na.rm=T)
}

# Calculate max drawdown
Max.Drawdown(Eq) # Test stat from single sample
```

```
## [1] -0.3833982
```

Bootstrap Approach

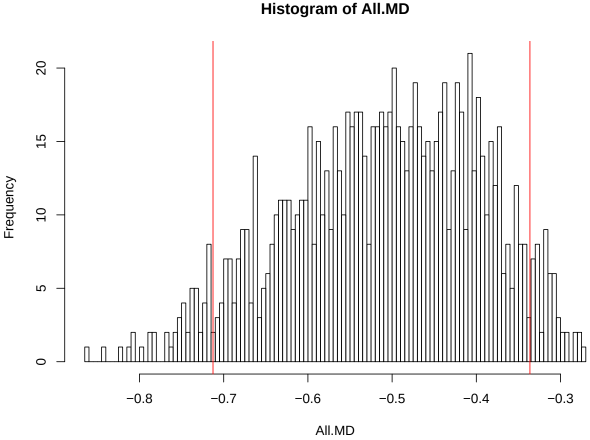
```r
#Bootstrap
Rets<-na.omit(diff(log(Eq)))
M<-1000; N<-length(Rets)
All.MD<-vector()
for (i in 1:M){ #Sample M times
  sampled.rets<-sample(x=Rets,size=N,replace=T) #Sample from rets
  sampled.eq<-exp(cumsum(sampled.rets)) # Construct equity curve
  MD<-Max.Drawdown(sampled.eq)# Calculate test stat
  All.MD<-c(All.MD,MD)}

#Calculate confidence interval
Conf.Int<-quantile(All.MD,probs=c(0.05,0.95))
Conf.Int
```

```
##          5%        95%
## -0.7126940 -0.3363451
```

Bootstrap Approach

```
# Plot sampling distribution
hist(All.MD,100);abline(v=Conf.Int,col=2)
```



Bootstrap Approach

- If returns are iid, then we can perform the bootstrap by resampling (with replacement) of individual returns. If the returns are serially correlated, then we should modify the basic bootstrap method and use **block bootstrap**
- In this case, a simple case or residual resampling will fail, as it is not able to replicate the correlation in the data. The block bootstrap tries to replicate the correlation by resampling instead blocks of data.
- Methods:
  - Simple Block Bootstrap: Split the observed sample into non-overlapping blocks. Randomly sample with replacement among blocks.
  - Moving Block Bootstrap: Split observed sample into $n - b + 1$ overlapping blocks of length b, i.e. observation 1 to b will be block 1, observation 2 to $b + 1$ will be block 2 etc. Then from these $n - b + 1$ blocks, n/b blocks will be drawn at random with replacement. Then aligning these n/b blocks in the order they were picked, will give the bootstrap observations.

Testing for Skill vs Luck

Example

- You design a trend following strategy, and backtest it on S&P500 Index for the period 2010-2016
- Your backtest results show that total return from the strategy is 70% !!
- You apply t-test to daily returns, and found that they are statistically significantly different from zero at 99% confidence level
- You think that you are genius and found the way to become rich
- Then you realize that, S&P500 Index experienced a long-lasting bullish period for the same time period and the total return for the buy&hold portfolio is 100% !!
- Thus for that time period, any **random** strategy that has a long bias may yield superior absolute returns
- Thus obtaining a good backtest performance does not always suggests skill. We should explicitly test for skill vs luck
- In an uptrend, we want to see the following relative ranking for the performance of trading strategies:
  - Any random model < Any random model with a long bias < Our system (i.e. an intelligent system with a long bias)

Sequential Testing

- To differentiate between skill vs luck, test your strategy against:
  - Any random strategy
  - Any random strategy with same constraints (e.g. long-only, same position sizing method, etc)
  - Any random strategy with same constraints and with same trade characteristics (e.g. **actual** trade direction bias, trade duration, etc)

Monte Carlo Approach

Testing Againts Any Random Strategy

- H_0 : Our strategy is no better than any random trading strategy
- Construct the sampling distribution under H_0:
  - Construct a random trading strategy
    - For each day in our backtest period, randomly draw the direction of strategy positions, i.e. randomly draw from set $\{-1, 0, 1\}$ where -1 represents a short position, 0 represents a neutral position and $+1$ represents a long position. This step gives us the strategy positions, Pos_t.
    - For simplicity, we assume a unit position sizing. This can also be randomized.
    - Among daily returns for the **underlying asset** (not the strategy), randomly select a permutation (i.e. reshuffle the daily returns of the underlying asset to randomly change their ordering). This step gives us the reshuffled returns, $\tilde{r}_t$.
    - Multiply the positions with the reshuffled returns to obtain the strategy returns, $r_t^{Strategy} = Pos_t \tilde{r}_t$
    - Calculate the performance statistics that we are interested in
  - Perform the above step many times and obtain the sampling distribution

Monte Carlo Approach

Testing Againts Random Strategies With Same Constraints

- H_0 : Our strategy is no better than any random trading strategy with same constraints
- Example: Our strategy is a long-only strategy, i.e. our constraint is that the positions can be either long or neutral
- Construct the sampling distribution under H_0:
  - Construct a random trading strategy
    - For each day in our backtest period, randomly draw the direction of strategy positions, i.e. randomly draw from set $\{0, 1\}$ This step gives us the strategy positions, Pos_t.
    - For simplicity, we assume a unit position sizing. This can also be randomized.
    - Among daily returns for the **underlying asset** (not the strategy), randomly select a permutation (i.e. reshuffle the daily returns of the underlying asset to randomly change their ordering). This step gives us the reshuffled returns, $\tilde{r}_t$.
    - Multiply the positions with the reshuffled returns to obtain the strategy returns, $r_t^{Strategy} = Pos_t \tilde{r}_t$
    - Calculate the performance statistics that we are interested in
  - Perform the above step many times and obtain the sampling distribution

Monte Carlo Approach

Testing Againts Random Strategies With Same Constraints and Same Trade Characteristics

- H_0 : Our strategy is no better than any random trading strategy with same constraints and same trade characteristics
- Example: We are testing against random long-only strategies with same number of long/neutral days
- Construct the sampling distribution under H_0:
    - Construct a random trading strategy
        - Keep the positions of our original strategy Pos_t^{Orig}. You can shuffle the ordering.
        - Among daily returns for the **underlying asset** (not the strategy), randomly select a permutation (i.e. reshuffle the daily returns of the underlying asset to randomly change their ordering). This step gives us the reshuffled returns, $\tilde{r}_t$.
        - Multiply the positions with the reshuffled returns to obtain the strategy returns, $r_t^{Strategy} = Pos_t^{Orig}\tilde{r}_t$
        - Calculate the performance statistics that we are interested in
    - Perform the above step many times and obtain the sampling distribution

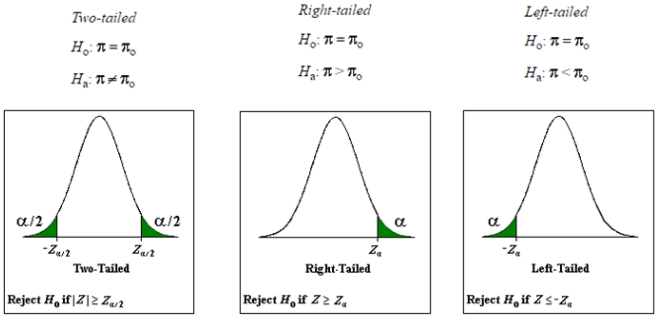
Monte Carlo Approach

Steps for hypothesis testing:

- Define the hypotheses:
  - H_0: Hypothesis that we want to reject (null hypothesis)
  - H_A: Hypothesis that we want to reject (alternative hypothesis)
- Obtain the sampling distribution under the assumption that null hypothesis is true
- Determine the confidence level for the test, α
- Determine the rejection region
  - See next slide
- Check whether the test (i.e. sample) statistics is within rejection region or not:
  - If so, reject the null hypothesis
  - Otherwise, fail to reject the null hypothesis
- p-value: Probability of observing such a test statistics if the null hypothesis is true

Monte Carlo Approach

Steps for hypothesis testing:



Monte Carlo Approach

- Assume that we want to test whether the sharpe ratio of our strategy is statistically significantly better than a random strategy with same constraints and same trade characteristics
- Using the above procedures, we can construct the sampling distribution of sharpe ratios under the assumption than null hypothesis is true
- Then we can perform a hypothesis test for our trading strategy performance
  - Check whether the sharpe ratio that we obtained in backtesting is within the rejection region or not (use a right tailed test)
  - **p-value:** Probability of observing such a performance (i.e. sharpe ratio) if your model is worthless

Monte Carlo Approach

```
# Underlying rets (Positive lag for lag.xts)
Rets<-na.omit(Cl(BIST)/lag(Cl(BIST),+1)-1)

# Monte carlo
# Positions: Original positions from our strategy
M<-1000; N<-length(Rets)
All.SR<-vector()
for (i in 1:M){ #Sample M times
  sampled.rets<-sample(x=Rets,size=N,replace=T) #Sample from rets
  sampled.strategy.rets<-na.omit(sampled.rets*Positions)
  SR<-sqrt(260)*mean(sampled.strategy.rets)/
        sd(sampled.strategy.rets)# Calculate test stat
  All.SR<-c(All.SR,SR)}
```

Monte Carlo Approach

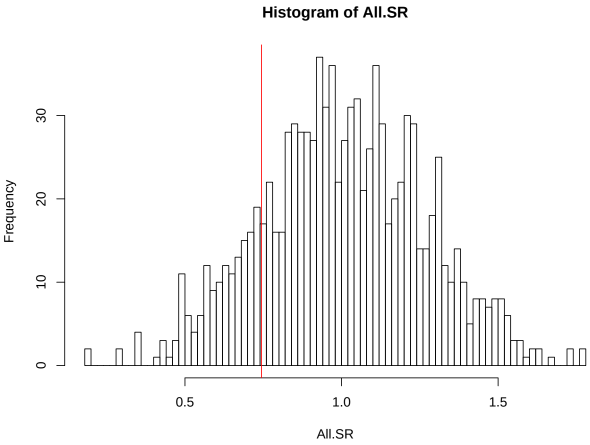
```
#Calculate the sharpe ratio for the strategy
Ret.Orig<-na.omit(Eq/lag(Eq,+1)-1)
SR.Orig<-sqrt(260)*mean(Ret.Orig)/sd(Ret.Orig)

# Calculate the p-value (right tailed test!!)
length(which(All.SR>SR.Orig))/length(All.SR)
```

```
## [1] 0.836
```

Monte Carlo Approach

```
# Plot sampling distribution
hist(All.SR,100);abline(v=SR.Orig,col=2)
```



Monte Carlo Approach

- In this example, we test whether the sharpe ratio of our strategy (which was 0.75) is superior to a random strategy with same constraints and same trading characteristics
- We fail to reject the null hypothesis with a p-value of 0.83
- Thus the sharpe ratio of our strategy is no better than a random strategy with same constraints and same trading characteristics